

When Does Episodic Memory Help or Hurt LLM Agents? A Two-Environment Comparative Study

Anonymous ACL submission

Abstract

LLM-based agents are increasingly augmented with episodic memory, yet it remains unclear *under what conditions* such memory improves performance. We investigate this through a controlled study across two interactive environments: ALFWorld (structured household tasks, fixed action grammar) and WebShop (open-ended e-commerce, free-form navigation). Our memory-centric framework stores outcome-validated episodic experiences and retrieves them via two complementary mechanisms: proactive *Context Retrieval* (CR) and on-demand hybrid *Tool Retrieval* (TR). On ALFWorld, CR+TR improves over the ReAct baseline by up to +19.4pp, approaching Reflexion at one-third the step cost. On WebShop, the *same* system *degrades* performance by up to -30.5pp. A hard-negative ablation reveals that irrelevant memories (-12.5pp) cause *less* harm than semantically “relevant” ones (-30.5pp), indicating active misdirection rather than noise. In our experiments, we attribute this ~47pp divergence to three environment properties: *task compositionality*, *action space regularity*, and *learning transfer radius*. We distill these observations into a diagnostic framework for when episodic memory is likely to help or hurt — though broader validation across additional environments and models remains necessary before these properties can be treated as general predictors.

1 Introduction

LLM-based agents are increasingly augmented with episodic memory: systems that store past experiences and retrieve them to guide future decisions. Recent work has demonstrated impressive gains from such augmentation (Voyager (Wang et al., 2023), GITM (Zhu et al., 2023), MemoryBank (Zhong et al., 2024), and Generative Agents (Park et al., 2023)) under the implicit assumption that storing and retrieving past experience will improve performance. But does it always?

We investigate this question through a controlled comparative study across two interactive environments: ALFWorld (Shridhar et al., 2021), a structured household task environment with compositional goals and a fixed action grammar, and WebShop (Yao et al., 2022), an open-ended e-commerce environment with free-form navigation. In our experiments, the answer depended critically on environment properties. On ALFWorld, our memory system improves over the ReAct (Yao et al., 2023) baseline by up to +19.4pp on unseen tasks, nearly matching Reflexion (Shinn et al., 2023) (+21.6pp) at roughly one-third the step cost. On WebShop, the *same* memory system *degrades* performance by up to 30.5pp. Strikingly, a hard-negative control that retrieves maximally *irrelevant* experiences (-12.5pp) outperforms the properly configured system (-30.5pp), revealing that semantically similar but non-transferable memories are more harmful than obviously irrelevant ones in this setting. This ~47pp swing, consistent across development and held-out splits, is the central empirical finding of this paper.

Our analysis attributes this divergence to three environment properties: *task compositionality* (shared sub-goals vs. unique tasks), *action space regularity* (fixed grammar vs. free-form), and *learning transfer radius* (one learning benefiting many tasks vs. near-zero reuse). In the two environments we study, when all three favored transfer, memory yielded substantial gains; when they did not, retrieved experiences actively misled the agent. Our results suggest that high-similarity retrieval alone does not guarantee performance gains, as retrieved trajectories may lack transferability to the current task — though the generality of this pattern awaits validation in further settings. This points to a potentially important distinction between retrieval relevance and actionable utility, one that deserves greater attention in future work on memory-augmented agents. Accordingly, the novelty of this

work is primarily empirical and diagnostic rather than proposing a fundamentally new memory architecture, we use a controlled episodic-memory setup to study when retrieval transfer helps and when it fails.

In summary, our contributions are:

1. A **controlled empirical study** showing opposite effects of the same memory system: +19.4pp on ALFWorld vs. -30.5pp on WebShop, with hard-negative ablations revealing that “relevant” but non-transferable memories cause more harm than irrelevant ones.
2. A **diagnostic framework for retrieval transfer**, based on three environment properties: task compositionality, action space regularity, and learning transfer radius, that characterize when episodic memory is likely to help or hurt LLM agents.
3. A **memory-centric agent framework** with outcome-validated episodic memory and two retrieval mechanisms (proactive CR and on-demand TR), evaluated across two diverse environments.

2 Related Work

Tool-augmented and memory-augmented agents. ReAct (Yao et al., 2023) interleaves reasoning with environment actions, and Reflexion (Shinn et al., 2023) extends this with iterative self-correction. Several systems augment LLM agents with persistent memory: Voyager (Wang et al., 2023) builds a verified skill library in Minecraft, GITM (Zhu et al., 2023) stores successful goal decompositions, Generative Agents (Park et al., 2023) use a memory stream with reflection, MemoryBank (Zhong et al., 2024) incorporates forgetting curves, A-Mem (Xu et al., 2025) treats memory itself as an agentic module that autonomously decides what to store, update, and retrieve, and EvoSkill (Alzubi et al., 2026) automatically discovers and evolves reusable skills from execution failures. A common thread is the assumption that accumulated knowledge, whether stored as memories or skills, is net-positive. Our work challenges this by showing that the same episodic memory mechanism yields +19pp gains in one environment and -31pp degradation in another.

Interactive benchmarks. ALFWorld (Shridhar et al., 2021) provides compositional household tasks with a fixed action grammar and shared sub-goal structure, making past experience highly reusable. WebShop (Yao et al., 2022) simulates e-commerce with free-form search and unique products, where successful strategies rarely transfer. This structural contrast motivates our comparative design.

Retrieval-augmented generation. RAG (Lewis et al., 2020) improves outputs by conditioning on retrieved documents, and has been extended to agentic settings with trajectory or demonstration retrieval. While the RAG literature focuses on improving retrieval quality under the assumption that relevant retrieval helps, our results reveal a complementary failure mode: when retrieved experiences are *superficially* similar but *structurally* non-transferable, retrieval actively misleads the agent. To our knowledge, this is the first work to systematically characterize when episodic memory helps versus hurts LLM agent performance, providing a diagnostic framework based on task compositionality, action space regularity, and learning transfer radius.

3 Methodology

We evaluate a unified memory-centric framework across two interactive environments with contrasting structural properties.

3.1 Environments

ALFWorld. ALFWorld (Shridhar et al., 2021) provides a text-based interface to the ALFRED embodied household benchmark. Agents issue actions from a fixed grammar of ~15 verb templates (e.g., go to, take, open, heat). Six task types share a compositional goal structure: SEARCH → ACQUIRE → TRANSFORM → PLACE. The small object vocabulary (~50 types) and shared sub-goal phases yield high structural overlap between tasks.

WebShop. WebShop (Yao et al., 2022) is an e-commerce navigation environment where agents find and purchase products matching natural-language specifications via `search[query]` and `click[element]` actions. Each task targets a unique product, resulting in minimal structural overlap and an effectively infinite action space.

3.2 Base Agent: ReAct

224

All agents build on ReAct (Yao et al., 2023), interleaving reasoning traces with actions. We use Gemini 2.5 Flash as the base LLM for all experiments.

3.3 Memory Systems

Our framework augments ReAct with a persistent knowledge base and two retrieval mechanisms: proactive *Context Retrieval* (CR) at task start and on-demand *Tool Retrieval* (TR) during execution. We emphasize that Context Retrieval (CR) and Tool Retrieval (TR) address complementary aspects of transfer: CR improves state understanding through contextual grounding, while TR enables action-level reuse. Their combination is designed to balance interpretability and execution efficiency.

3.3.1 Knowledge Base Construction

The knowledge base is constructed offline from training trajectories using LLM-based extraction. For each task, we extract *issue-learning pairs*: structured records associating a mistake with its successful correction. Each entry includes task metadata, the issue-learning texts, a validation level reflecting downstream outcome (VALID_NEXT_TRIAL, VALID_SAME_TRIAL, or CANDIDATE), and provenance information. We assign scalar validation weights ω to each level: $\omega_{\text{next}} = 1.0$, $\omega_{\text{same}} = 0.9$, $\omega_{\text{cand}} = 0.6$, used by both retrieval mechanisms to prioritize high-confidence learnings. Full schema details are in Appendix B.

3.3.2 Context Retrieval (CR)

CR operates proactively: before the agent acts, it retrieves relevant past learnings based on embedding similarity (using `gemini-embedding-001`) between the new task description and historical tasks. The top- k ($k=5$) most similar tasks are identified, their associated learnings are pooled and scored by:

$$\text{Score}_{\text{CR}}(\ell) = \text{sim}(\mathbf{v}_q, \mathbf{v}_{t(\ell)}) \times \omega_\ell \quad (1)$$

where sim is cosine similarity and ω_ℓ is the validation weight.

Dynamic context sizing. Rather than retrieving a fixed number of learnings, CR adapts the context window to neighborhood complexity. Let L_{max} denote the maximum learning count among the

top- k tasks. The retrieval quota is:

$$N_{\text{pick}} = \min(\lceil \alpha \cdot L_{\text{max}} \rceil, p) \quad (2)$$

where $\alpha = 1.5$ is a scaling factor and $p \in [5, 25]$ is a tunable cap. The top-scored learnings are formatted as issue-learning pairs and prepended to the agent’s prompt. The full procedure is given in Algorithm 1 (Appendix A).

3.3.3 Tool Retrieval (TR)

TR provides on-demand retrieval exposed as an action `help["issue"]`. It uses hybrid dense-sparse retrieval: cosine similarity over issue embeddings captures semantic matches, while BM25 (Robertson and Zaragoza, 2009) over concatenated fields captures lexical matches. The candidate pool is the union of the top-100 results from each strategy. Scores are normalized to $[0, 1]$ and fused:

$$\text{Score}_{\text{TR}}(e) = (0.7 \cdot \text{cos}_{\text{norm}} + 0.3 \cdot \text{BM25}_{\text{norm}}) \times \omega_e \quad (3)$$

The top- k ($k=3$) results are returned as issue-solution pairs. The full procedure is given in Algorithm 2 (Appendix A).

3.3.4 Hard Negative Control

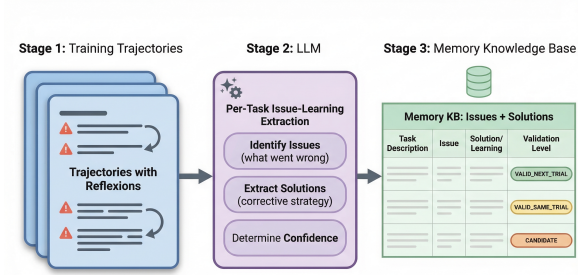
To verify that gains stem from retrieval *relevance* rather than additional prompt context, we introduce a hard negative control that inverts the selection criterion: CR retrieves the *least* similar tasks and TR selects from the bottom of each retrieval strategy, producing prompts with the same number of learnings but from maximally irrelevant tasks.

3.4 Reflexion Baseline

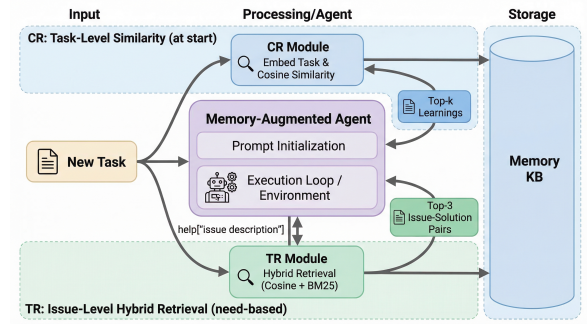
We compare against Reflexion (Shinn et al., 2023), an iterative self-correction framework allowing up to 7 trials per task. Unlike our *cross-task* memory (learnings from other tasks), Reflexion is *intra-task* (retrying the same task with accumulated self-critiques), at substantially higher computational cost.

4 Experimental Setup

All agent reasoning uses Gemini 2.5 Flash, and all retrieval embeddings use Google’s `gemini-embedding-001`, held constant across experiments.



(a) Offline KB construction from training trajectories.



(b) Inference-time retrieval via CR and TR.

Figure 1: Overview of the episodic memory framework. (a) Training trajectories with reflexions are processed by an LLM to extract per-task issue–learning pairs with validation levels. (b) Context Retrieval (CR) provides task-level learnings at the start; Tool Retrieval (TR) provides issue-level solutions on demand during execution.

Tasks and splits. For ALFWorld, the knowledge base is constructed from *alfworld-mini*, a curated subset of 120 training games (20 per task type). We evaluate on the standard validation-seen (140 tasks) and validation-unseen (134 tasks) splits. For WebShop, we evaluate on development (200 tasks) and held-out test (200 tasks) splits, with the knowledge base extracted from separate training tasks. Full dataset details are in Appendix C.

Framework variants. We evaluate six variants per environment: ReAct (baseline), ReAct + Reflexion (up to 7 trials), ReAct + CR, ReAct + TR, ReAct + CR + TR, and a hard-negative ablation control. All memory variants execute a single trial per task.

Metrics. We report accuracy (binary success rate), average steps per trial and per task (capturing Reflexion’s cumulative retry cost), and gain from the ReAct baseline in percentage points. Full metric definitions are in Appendix C. All hyperparameters and implementation details are in Appendix C.4 and C.5.

5 Results

Tables 1–4 report accuracy, step counts, success ratios, and gain from the ReAct baseline. The central finding: the same memory system produces substantial gains in ALFWorld and substantial losses in WebShop.

5.1 ALFWorld: Memory Helps

We ablate the memory system by evaluating CR, TR, and their combination independently.

Context Retrieval drives large gains. CR alone raises accuracy by +15.7pp (seen) and +14.2pp

(unseen), with consistent gains across splits suggesting generalizable patterns rather than memorized solutions. CR also *reduces* average steps, indicating that proactive guidance makes the agent more efficient.

Tool Retrieval complements CR, especially on unseen tasks. TR alone yields a modest +2.1pp on seen tasks but +8.2pp on unseen tasks, suggesting that on-demand issue-level retrieval is most valuable for genuinely novel situations.

CR+TR approaches Reflexion at one-third the cost. The combined CR+TR achieves +16.4pp/+19.4pp (seen/unseen), nearly matching Reflexion’s +20.7pp/+21.6pp at roughly one-third the step cost (23 vs. 75–79 steps/task).

Hard-negative control confirms relevance matters. Replacing relevant retrievals with maximally irrelevant ones degrades performance to −5.7pp (seen) and −17.2pp (unseen), ruling out prompt-length effects as the source of memory gains.

5.2 WebShop: Memory Hurts

All memory variants degrade performance. CR reduces accuracy by −29.0pp (dev), TR by −21.5pp, and CR+TR by −30.5pp. Adding more retrieved context produces monotonically worse outcomes, suggesting retrieved learnings introduce spurious heuristics from structurally dissimilar tasks.

The hard-negative paradox. Strikingly, the hard-negative variant (−12.5pp dev, −15.0pp test) *outperforms* all properly configured retrieval variants. We hypothesize that obviously irrelevant retrievals are easier for the agent to recognize

Table 1: ALFWorld validation-seen results (140 tasks). CR = Context Retrieval, TR = Tool Retrieval. **Bold** = best accuracy.

Framework	Success / Total	Accuracy	Steps/Trial	Steps/Task	Gain
ReAct	86 / 140	61.43	24.29	24.29	—
ReAct + Reflexion [†]	115 / 140	82.14	31.72	79.06	+20.71
ReAct + CR	108 / 140	77.14	23.09	23.09	+15.71
ReAct + TR	89 / 140	63.57	26.90	26.90	+2.14
ReAct + CR + TR	109 / 140	77.86	25.41	25.41	+16.43
ReAct + hard-neg CR + TR*	78 / 140	55.71	28.41	28.41	−5.71

[†]Max 7 trials; Steps/Task reflects cumulative cost. *Ablation control retrieving *least*-similar entries.

Table 2: ALFWorld validation-unseen results (134 tasks). CR = Context Retrieval, TR = Tool Retrieval. **Bold** = best accuracy.

Framework	Success / Total	Accuracy	Steps/Trial	Steps/Task	Gain
ReAct	83 / 134	61.94	22.38	22.38	—
ReAct + Reflexion [†]	112 / 134	83.58	30.41	74.90	+21.64
ReAct + CR	102 / 134	76.12	22.55	22.55	+14.18
ReAct + TR	94 / 134	70.15	24.93	24.93	+8.21
ReAct + CR + TR	109 / 134	81.34	23.35	23.35	+19.40
ReAct + hard-neg CR + TR*	60 / 134	44.78	33.01	33.01	−17.16

[†]Max 7 trials; Steps/Task reflects cumulative cost. *Ablation control retrieving *least*-similar entries.

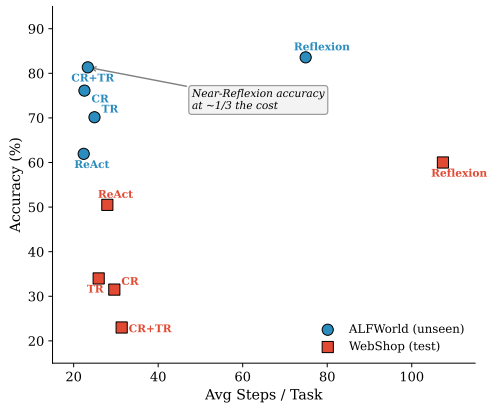


Figure 2: Accuracy vs. computational cost (steps/task) on held-out splits. CR+TR achieves near-Reflexion accuracy on ALFWorld at $\sim 1/3$ the cost. On WebShop, memory degrades accuracy without reducing cost.

and ignore, whereas semantically similar but non-transferable experiences are treated as trustworthy guidance, producing worse outcomes through confident misdirection.

Only Reflexion improves, at high cost. Reflexion achieves +11.0pp (dev) and +9.5pp (test) as the only variant surpassing baseline, but at 3.5–3.8 $\times$ the step cost. Its intra-task nature circumvents the transfer problem entirely.

Test-set confirmation. All findings replicate on held-out test splits (Table 4), confirming that the observed patterns are robust.

5.3 Cross-Environment Comparison

The same memory system, base LLM, and knowledge base pipeline produce a ~ 47 pp swing in effect size: CR+TR yields +19.4pp on ALFWorld unseen versus −27.5pp on WebShop test. This divergence is consistent across both split pairings (46.9pp gap in each), confirming it is attributable to environment properties rather than evaluation-set idiosyncrasies.

Figure 3 visualizes this divergence directly: on ALFWorld, all memory variants (CR, TR, CR+TR) exceed the ReAct baseline (dashed line), with CR+TR approaching Reflexion’s accuracy. On WebShop, the pattern inverts: every memory variant falls below the baseline, while Reflexion remains the sole method that improves over ReAct. The mirrored structure across paired splits (seen/dev and unseen/test) confirms that the effect is systematic rather than split-specific.

Figure 4 quantifies these effects as gain from the ReAct baseline across all four evaluation splits. The contrast is stark: ALFWorld variants cluster in the positive region (green/blue), while Web-

Table 3: WebShop development set results (200 tasks). CR = Context Retrieval, TR = Tool Retrieval. **Bold** = best accuracy.

Framework	Success / Total	Accuracy	Steps/Trial	Steps/Task	Gain
ReAct	112 / 200	56.00	24.44	24.44	—
ReAct + Reflexion [†]	134 / 200	67.00	24.64	85.13	+11.00
ReAct + CR	54 / 200	27.00	29.39	29.39	−29.00
ReAct + TR	69 / 200	34.50	23.51	23.51	−21.50
ReAct + CR + TR	51 / 200	25.50	30.52	30.52	−30.50
ReAct + hard-neg CR + TR [*]	87 / 200	43.50	28.66	28.66	−12.50

[†]Max 7 trials; Steps/Task reflects cumulative cost across all trials. ^{*}Ablation control retrieving *least*-similar entries.

Table 4: WebShop test set results (200 tasks). CR = Context Retrieval, TR = Tool Retrieval. **Bold** = best accuracy.

Framework	Success / Total	Accuracy	Steps/Trial	Steps/Task	Gain
ReAct	101 / 200	50.50	27.97	27.97	—
ReAct + Reflexion [†]	120 / 200	60.00	26.81	107.38	+9.50
ReAct + CR	63 / 200	31.50	29.62	29.62	−19.00
ReAct + TR	68 / 200	34.00	25.92	25.92	−16.50
ReAct + CR + TR	46 / 200	23.00	31.36	31.36	−27.50
ReAct + hard-neg CR + TR [*]	71 / 200	35.50	31.43	31.43	−15.00

[†]Max 7 trials; Steps/Task reflects cumulative cost across all trials. ^{*}Ablation control retrieving *least*-similar entries.

Shop variants cluster in the negative region (orange/red). Notably, the hard-negative control on WebShop (−12.5/−15.0pp) sits *above* the properly configured retrieval variants (−30.5/−27.5pp), visually highlighting the paradox that relevant retrieval causes more harm than irrelevant retrieval in low-transfer environments.

The hard-negative controls reveal an asymmetry: on ALFWorld, inverting retrieval degrades accuracy by 22–37pp relative to correct CR+TR; on WebShop, the hard-negative *outperforms* correct retrieval by 12–18pp. Reflexion succeeds in both environments (+21pp ALFWorld, +10pp WebShop) because intra-task self-correction does not depend on cross-task transfer, but at 3–4× the step cost.

6 Analysis and Discussion

We trace the divergence between environments to three structural properties and synthesize them into a diagnostic framework.

6.1 Task Compositionality

ALFWorld tasks decompose into shared sub-goal phases (SEARCH → ACQUIRE → TRANSFORM → PLACE) that recur across task types, so a modest set of phase-specific learnings covers a combinatorially large task space. WebShop tasks are

monolithic: each targets a unique product through a unique catalog path with no shared sub-task decomposition. This explains why CR yields +15.7pp on ALFWorld but −29pp on WebShop, and why the hard-negative paradox emerges: in WebShop, obviously irrelevant retrievals cause less harm than plausible-looking but non-transferable ones.

Principle. Episodic memory is most effective when tasks share compositional sub-task structure.

6.2 Action Space Regularity

ALFWorld’s fixed grammar of ~15 verb templates ensures that learned action sequences generalize across tasks. WebShop’s action space is syntactically simple but combinatorially vast: free-form search queries and dynamically generated page elements mean that specific navigation paths never recur. TR’s issue-level guidance accordingly helps in ALFWorld (+8.2pp unseen) but hurts in WebShop (−21.5pp).

Principle. Episodic memory is most effective when the action space is constrained and regular.

6.3 Learning Transfer Radius

We define the *learning transfer radius* as the number of distinct future tasks that benefit from a single learned experience. In ALFWorld, a learning

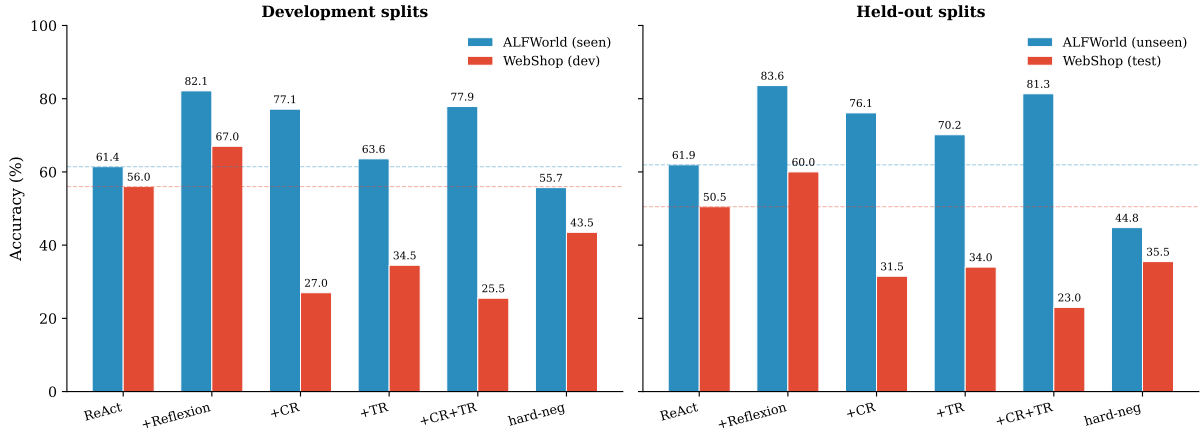


Figure 3: Accuracy across framework variants with paired splits: development (ALFWorld seen vs. WebShop dev) and held-out (ALFWorld unseen vs. WebShop test). Dashed lines indicate ReAct baselines.

like “open the container before taking an object” applies to dozens of tasks across multiple types (high radius). In WebShop, each learning is tied to a unique product (radius near zero). The empirical signature is visible in ALFWorld’s stronger unseen-split gains (+19.4pp vs. +16.4pp on seen), indicating effective cross-task generalization.

Principle. Episodic memory is most effective when the learning transfer radius is high.

6.4 Efficiency–Accuracy Trade-off

Figure 2 plots accuracy against computational cost (steps per task) on held-out splits, revealing a key practical distinction. On ALFWorld, CR+TR occupies an efficient frontier: it achieves near-Reflexion accuracy (+19.4pp vs. +21.6pp) at roughly one-third the step cost (~ 23 vs. ~ 79 steps/task), making it the preferred choice when compute budgets are constrained. On WebShop, memory variants shift the agent into a strictly dominated region: accuracy degrades without any compensating reduction in cost. Reflexion improves accuracy in both environments but consistently incurs 3–4 $\times$ the baseline step cost.

6.5 Diagnostic Framework

Table 5 summarizes how the two environments differ along these dimensions.

When all three properties favor transfer, memory provides substantial gains; when they do not, retrieved experiences actively mislead the agent. Before deploying a memory system, practitioners should assess their target environment along these dimensions: task compositionality, action space regularity, and learning transfer radius. When

Table 5: Diagnostic framework for episodic memory effectiveness. Effect sizes relative to ReAct baseline (seen/unseen for ALFWorld, dev/test for WebShop).

Property	ALFWorld	WebShop
Task Compositionality	High (shared sub-goals)	Low (unique products)
Action Space Regularity	High (fixed, ~ 15 verbs)	Low (free-form)
Learning Transfer Radius	High (1 $\rightarrow$ many tasks)	Near-zero
Memory Effect (CR+TR)	+16.4/+19.4pp	−30.5/−27.5pp
Hard-neg Effect*	−5.7/−17.2pp	−12.5/−15.0pp

*Hard-neg *outperforms* proper retrieval on WebShop (−12.5 vs. −30.5pp), indicating semantic similarity is counterproductive when transfer radius is near zero.

these properties are unfavorable, alternatives such as intra-task self-correction or memory systems with explicit gating mechanisms may be more appropriate.

7 Limitations

We evaluate only one base LLM and two environments. All memory variants use single-trajectory evaluation, and we do not include repeated-run confidence estimates. These constraints limit how far the proposed diagnostic framework can be generalized: the three environment properties (task compositionality, action space regularity, and learning transfer radius) are derived from and validated on only two data points, so they should be treated as preliminary hypotheses rather than established predictors. Future work should test additional models, domains, and multi-run evaluation protocols to assess whether these properties hold as reliable indicators of memory benefit more broadly.

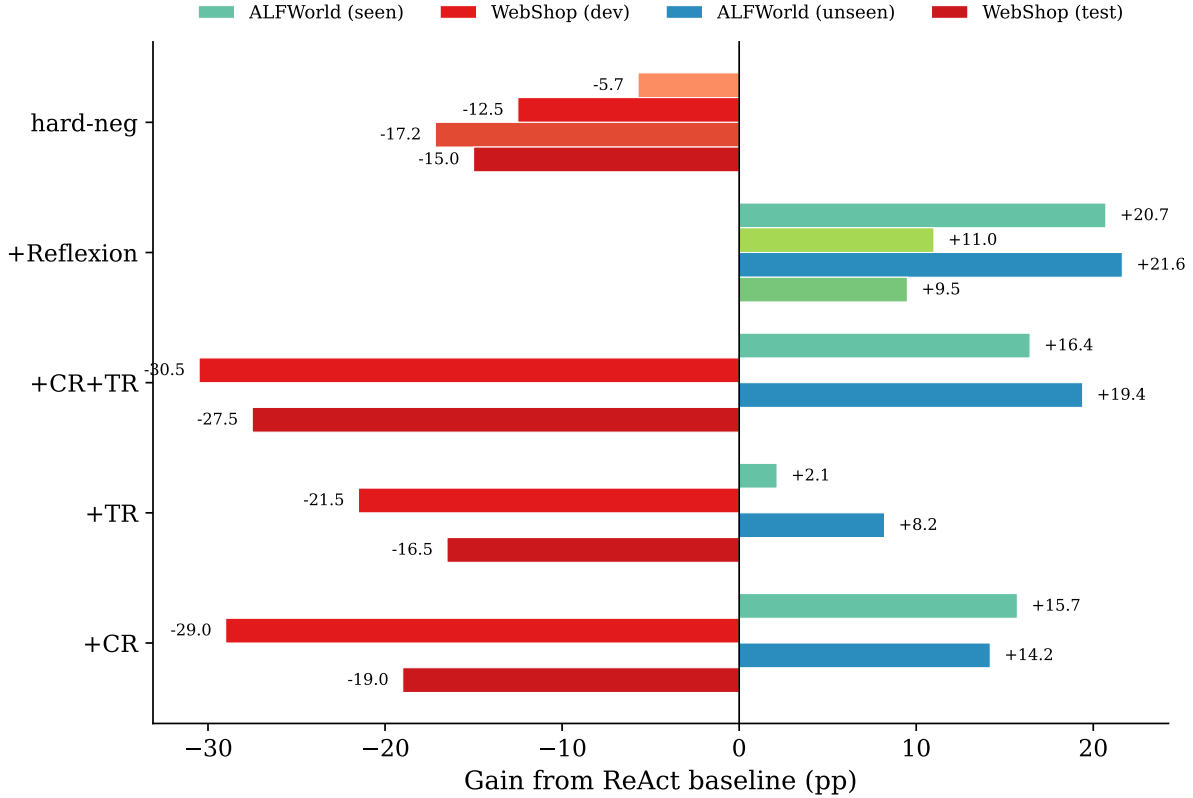


Figure 4: Gain from ReAct baseline (percentage points) for each memory variant across all four evaluation splits. Green/blue indicates improvement; orange/red indicates degradation. Development splits (ALFWorld seen, WebShop dev) and held-out splits (ALFWorld unseen, WebShop test) show consistent patterns, confirming that the ~ 47 pp swing between environments is robust.

8 Conclusion

This work provides evidence that episodic memory is not a universal enhancement for LLM-based agents. Through a controlled study applying an identical memory system to ALFWorld and WebShop, we observe that the same retrieval mechanisms produce a ~ 47 pp swing in effect: up to $+19.4$ pp improvement versus -30.5 pp degradation. Our hard-negative ablation reveals that in the low-transfer environment we studied, irrelevant memories caused *less* harm than semantically “relevant” ones, suggesting confident misdirection rather than noise injection as the failure mode — at least under these experimental conditions.

We attribute this divergence to three structural properties (task compositionality, action space regularity, and learning transfer radius) and contribute a diagnostic framework offering practitioners tentative criteria for anticipating when episodic memory may help or hurt. Because these criteria are grounded in two environments with a single base model, they should be treated as hypotheses to

test rather than universal rules. Future directions include adaptive memory gating that suppresses retrieval when transfer confidence is low, validation on additional domains and models, and memory filtering strategies that assess transferability before injecting retrieved content.

AI Disclosure. AI writing assistants were used to help draft and edit portions of this manuscript. All scientific content, experimental design, and analysis were conducted by the authors.

References

- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyan Chen, and Tu Vu. 2026. EvoSkill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*.
- Patrick Lewis, Ethan Perez, Aleksandara Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Joon Sung Park, Joseph C O’Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*.

Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, Yu Qiao, Zhaoxiang Zhang, and Jifeng Dai. 2023. Ghost in the Minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory. *arXiv preprint arXiv:2305.17144*.

Appendix

A Retrieval Algorithms

Algorithm 1 Context Retrieval from Knowledge Base

Require: Task description q ; knowledge base $\mathcal{K}$; neighbor count $k=5$; scaling factor $\alpha=1.5$; cap $p \in [5, 25]$; validation weights W

Ensure: Selected learnings C

```

1:  $\mathbf{v}_q \leftarrow \text{EMBED}(q)$ 
2:  $\mathcal{T}_{\text{top}} \leftarrow \text{Top-}k \text{ tasks in } \mathcal{K} \text{ by } \text{sim}(\mathbf{v}_q, \mathbf{v}_t) \text{ \{filter exact matches\}}$ 
3:  $L_{\text{max}} \leftarrow \max_{t \in \mathcal{T}_{\text{top}}} \text{LEARNINGCOUNT}(t)$ 
4:  $N_{\text{pick}} \leftarrow \min(\lceil \alpha \cdot L_{\text{max}} \rceil, p)$ 
5:  $\mathcal{L}_{\text{pool}} \leftarrow \emptyset$ 
6: for each  $(t, s_t) \in \mathcal{T}_{\text{top}}$  do
7:   for each learning  $\ell \in \text{GETLEARNINGS}(t)$  do
8:      $\text{score}(\ell) \leftarrow s_t \times W[\ell.\text{valid\_level}]$ 
9:      $\mathcal{L}_{\text{pool}} \leftarrow \mathcal{L}_{\text{pool}} \cup \{(\ell, \text{score}(\ell))\}$ 
10:  end for
11: end for
12: Sort  $\mathcal{L}_{\text{pool}}$  by score descending; deduplicate by learning text
13:  $C \leftarrow \text{first } N_{\text{pick}} \text{ elements of } \mathcal{L}_{\text{pool}}$ 
14: return  $C$ 
```

B Knowledge Base Schema

Each entry in the knowledge base is a structured record containing the following fields:

- **Task metadata:** task description, object type, action verbs, goal phase (SEARCH, ACQUIRE, TRANSFORM, PLACE, or RECOVER).
- **Issue-learning pair:** issue_text (the specific mistake encountered) and learning_text (the successful correction).
- **Validation level:** a confidence label reflecting downstream outcome:
 - VALID_NEXT_TRIAL: the learning was applied and led to success in a subsequent trial (highest confidence). Weight $\omega_{\text{next}} = 1.0$.
 - VALID_SAME_TRIAL: the learning is associated with success within the same trial. Weight $\omega_{\text{same}} = 0.9$.
 - CANDIDATE: the learning was generated but the task was not completed (lowest confidence). Weight $\omega_{\text{cand}} = 0.6$.
- **Provenance:** references to the source task,

Algorithm 2 Hybrid Tool Retrieval 616

Require: Issue description I_q ; knowledge base $\mathcal{K}$; result count $k=3$; dense weight $\alpha=0.7$; sparse weight $\beta=0.3$; validation weights W

Ensure: Top- k issue-learning pairs

```

1:  $q' \leftarrow \text{CLEAN}(I_q)$  {remove digits, normalize}
2:  $\mathbf{v}_q \leftarrow \text{EMBED}(q')$ 
3:  $\mathbf{s}_{\text{cos}} \leftarrow \text{COSINESIM}(\mathbf{v}_q, \mathcal{K}.\text{issue\_embeddings})$ 

4:  $\mathbf{s}_{\text{bm25}} \leftarrow \text{BM25}(q', \mathcal{K}.\text{index})$ 
5:  $\mathcal{C} \leftarrow \text{Top}(\mathbf{s}_{\text{cos}}, 100) \cup \text{Top}(\mathbf{s}_{\text{bm25}}, 100)$  {candidate pool}
6: for each  $i \in \mathcal{C}$  do
7:    $c_i \leftarrow \text{clamp}(\mathbf{s}_{\text{cos}}[i], 0, 1)$ 
8:    $b_i \leftarrow \text{MINMAXNORM}(\mathbf{s}_{\text{bm25}}[i], \mathcal{C})$ 
9:    $S_{\text{hybrid}} \leftarrow \alpha \cdot c_i + \beta \cdot b_i$ 
10:   $\text{score}(i) \leftarrow S_{\text{hybrid}} \times W[\mathcal{K}[i].\text{valid\_level}]$ 
11: end for
12: Sort  $\mathcal{C}$  by score descending
13: return first  $k$  elements of  $\mathcal{C}$ 

```

trial number, and step range for both the issue and its resolution.

C Experimental Details**C.1 Dataset Statistics**

ALFWorld. The full ALFWorld dataset (Shridhar et al., 2021) comprises 3,827 games split into a training set (3,553 games), a validation-seen set (140 games), and a validation-unseen set (134 games). For knowledge base construction, we use *alfworld-mini*, a curated subset of 120 training games (20 per task type $\times$ 6 task types). We run Reflexion on this subset and extract approximately 400 issue-learning entries from the resulting trajectories using the pipeline described in Section 3.3.1. The validation splits are identical to those of the full ALFWorld dataset, ensuring direct comparability with prior work. No training games appear in the validation sets.

WebShop. We partition the WebShop environment (Yao et al., 2022) into a development split (200 tasks) and a held-out test split (200 tasks). The knowledge base is extracted from Reflexion training trajectories on a separate set of training tasks with no overlap with either evaluation split, yielding approximately 677 issue-learning entries.

C.2 Framework Variants

We evaluate six framework variants in each environment:

1. **ReAct:** the base agent with no memory (Yao et al., 2023), serving as the primary baseline.
2. **ReAct + Reflexion:** the iterative self-correction baseline (Shinn et al., 2023), allowed up to 7 trials per task.
3. **ReAct + CR:** base agent augmented with Context Retrieval only.
4. **ReAct + TR:** base agent augmented with Tool Retrieval only.
5. **ReAct + CR + TR:** base agent augmented with both retrieval mechanisms.
6. **ReAct + hard-neg CR + TR:** ablation control that retrieves maximally *irrelevant* entries via inverted retrieval (Section 3.3.4).

All memory-augmented variants (CR, TR, CR+TR) execute a single trial per task, whereas Reflexion may execute up to 7 trials.

C.3 Evaluation Metrics

Accuracy (success rate). The fraction of tasks completed successfully, computed as a binary outcome per task.

Average Steps per Trial. The mean number of thought-action-observation steps per individual trial attempt. For Reflexion, this reflects the average length of a single trial, irrespective of retry count.

Average Steps per Task. The mean total steps per task across all trials. For single-trial methods, this equals Steps/Trial. For Reflexion, this accumulates steps across all retry trials (up to 7), capturing the full computational cost.

Gain from ReAct. The percentage-point difference between a variant’s accuracy and the ReAct baseline accuracy on the same split.

C.4 Hyperparameters

Table 6 lists all hyperparameters used across experiments.

Parameter	Value	Description
<i>Context Retrieval</i>		
p	[5, 25]	Tunable cap on retrieved learnings
α	1.5	Dynamic context sizing factor
k_{CR}	5	Neighbor tasks retrieved
<i>Tool Retrieval</i>		
k_{TR}	3	Results returned per query
Dense weight	0.7	Cosine similarity fusion weight
Sparse weight	0.3	BM25 fusion weight
Candidate pool	100+100	Top-100 from each strategy
<i>Validation Weights</i>		
ω_{next}	1.0	VALID_NEXT_TRIAL
ω_{same}	0.9	VALID_SAME_TRIAL
ω_{cand}	0.6	CANDIDATE
<i>Agent Configuration</i>		
Max steps/task	49	Episode length limit
Temperature	0.0	Deterministic decoding
Max output tokens	2,048	LLM generation limit
Max Reflexion trials	7	Retry budget

Table 6: Hyperparameters used across all experiments.

C.5 Implementation Details

All agent reasoning uses Gemini 2.5 Flash routed through the OpenRouter API. We use greedy decoding (temperature 0.0) with a maximum output length of 2,048 tokens. API calls employ exponential backoff (2 s–30 s) with up to 8 retry attempts.

Retrieval embeddings use gemini-embedding-001 via the Google GenAI SDK directly, with a batch size of 100 texts per API call. Embeddings are cached to disk alongside the knowledge base file; the cache includes a hash of the source file and the embedding model name for automatic invalidation on changes.

Each task trial is limited to 49 thought–action–observation steps across all agent variants. For Reflexion, this limit applies per trial, with up to 7 trials per task.